

the programmers protest too much when this is proposed, having two coworkers check each other's code is also a possibility.

9.7.3 Login Spoofing

In this insider attack, the perpetrator is a legitimate user who is attempting to collect other people's passwords through a technique called **login spoofing**. It is typically employed in organizations with many public computers on a LAN used by multiple users. Many universities, for example, have rooms full of computers where students can log onto any computer. It works like this. Normally, when no one is logged in on a UNIX computer, a screen similar to that of Fig. 9-32(a) is displayed. When a user sits down and types a login name, the system asks for a password. If it is correct, the user is logged in and a shell (and possibly a GUI) is started.



Figure 9-32. (a) Correct login screen. (b) Phony login screen.

Now consider this scenario. A malicious user, Mal, writes a program to display the screen of Fig. 9-32(b). It looks amazingly like the screen of Fig. 9-32(a), except that this is not the system login program running, but a phony one written by Mal. Mal now starts up the phony login program and walks away to watch the fun from a safe distance. When a user sits down and types a login name, the program responds by asking for a password and disabling echoing. After the login name and password have been collected, they are written away to a file and the phony login program sends a signal to kill its shell. This action logs Mal out and triggers the real login program to start and display the prompt of Fig. 9-32(a). The user assumes that she made a typing error and just logs in again. This time, however, it works. But in the meantime, Mal has acquired another (login name, password) pair.

9.8 OPERATING SYSTEM HARDENING

The single best way to handle security bugs is to not have them at all. Think of how nice it would be if we could accompany software with a mathematical proof that it is correct and contains no vulnerabilities. That is exactly what formal verification of software is all about. In the past, researchers have shown that it is actually

possible to verify a small operating system kernel against a formal specification to prove that processes are properly isolated (Klein et al. 2009). Frankly, this is incredibly cool. Others have applied formal methods to compilers and other programs.

An obvious limitation of formal verification is that it is only as good as the specification. If you make a mistake in the specification, the software may be vulnerable even though it was verified. Another problem is most of the proofs concern themselves solely with software, while assuming the hardware to be correct. As we saw earlier, hardware vulnerabilities make short work of such assumptions.

Besides hardware issues that leak information (e.g., via cache side channels or transient execution attacks), there are other hardware bugs that cause memory corruption. For instance, the circuits that encode bits in memory chips are packed so closely together, that reading or writing a value in one location in memory may interfere with the value in a location adjacent to it on the chip. Note that such locations are not necessarily close to each other in terms of the virtual or even physical addresses as seen by the software—DRAM memory may internally remap addresses to chip locations in wonderfully complex ways. By aggressively accessing one or a few locations in memory in quick repetition, the interference may build up and eventually cause a bit flip in the neighboring location. This sounds like magic, but yes, it is possible to change one value in memory (for instance, a value in the kernel) by reading another value at a completely unrelated address (for instance, in your own address space). The problem is known as the **Rowhammer vulnerability**. The exact nature of Rowhammer attacks is beyond this book and we will not discuss it further, except to add that, unfortunately, few formal software proofs take into account magic. For more information about it, see Kim et al. (2014), Konoth et al. (2018), Kim et al. (2020), and Hassan et al. (2021).

A more practical problem of the use of formal methods is that generating proofs for complex software is difficult to scale and massive software projects such as the Linux or Windows kernel are well beyond what we can achieve with formal verification. As a result, most of the software that we use today is riddled with vulnerabilities. Operating systems therefore protect themselves against attacks by means of software hardening.

9.8.1 Fine-Grained Randomization

We already discussed how randomization of the address space through Address Space Layout Randomization (ASLR) makes it difficult for attackers to find gadgets for their ROP attacks. Nowadays, all mainstream operating systems apply a form of ASLR. When applied to the kernel, it is known as KASLR.

Just how randomized is that kernel? The amount of randomness is called the entropy and is expressed in bits. Suppose an operating system kernel lives in an address range of 1 GB (2^{30} bytes) and is aligned to a 2 MB page boundary. The alignment means that the code can start at any address that is a multiple of the page

size of 2 MB (2^{21} bytes). Such a system will have $30 - 21 = 9$ bits available for randomization. In other words, the entropy is 9 bits. In other words, attackers need 512 guesses to find the kernel code. Suppose they find a vulnerability, such as a buffer overflow, that allows them to make the kernel jump to an address of their choosing and try out all possible values for these 9 bits, the system would (probably) crash on 511 attempts and hit the correct target once. Phrased differently, if the attackers can attack a few thousand machines, they have a pretty good chance of compromising the operating system kernel of at least some of them—even if the attack leaves a trail of crashes in its wake.

The entropy, or how *much* you randomize is not the only factor that determines the strength of randomization, it also matters *what* you randomize. KASLR implementations commonly use coarse-grained randomization, whereby the stack, code, and heap all start at a random location, but there is no randomization within these memory areas.

Fig. 9-33(a) shows an example. This is simple and fast. Unfortunately, it also means that it is sufficient for the attack to leak a single code pointer, say the start of one particular function, to break the randomization. All the other code will be at a fixed offset from this. More advanced randomization schemes randomize at a finer granularity. For instance, Fig. 9-33(b) shows a scheme where the locations of functions and heap objects are also randomized relative to each other. Instead of randomizing at the function level, it is also possible to do so at the page level, or even at the level of code fragments within a function. Now leaking a single code address is no longer sufficient, because it tells the attacker nothing about the location of other functions and code snippets. Fine-grained randomization also works for global data, data on the heap, and even local variables on the stack. It is even possible to rerandomize the locations of code and data every few seconds during program execution, thus greatly reducing the time available for an attacker to try to learn where things are (Giuffrida et al., 2013). The disadvantage of very fine-grained randomization is that shuffling things around hurts locality and increases fragmentation.

In general, KASLR is not considered a very strong defense against local attackers—that are able to run code locally on the machine. In fact, coarse-grained KASLR in particular has been broken many times.

9.8.2 Control-Flow Restrictions

Besides hiding the code at randomized locations, it is also possible to reduce the amount of code that the attacker can use. In particular, it would be nice if we could ensure that a return instruction can only return to the instruction following a function call, that a call instruction can only target a legitimate function, etc. Even if an attack overwrites a return address on the stack, the set of addresses to which it could divert the program's control flow is now much more limited. This idea became popular under the name **CFI (Control-Flow Integrity)**. It has been part

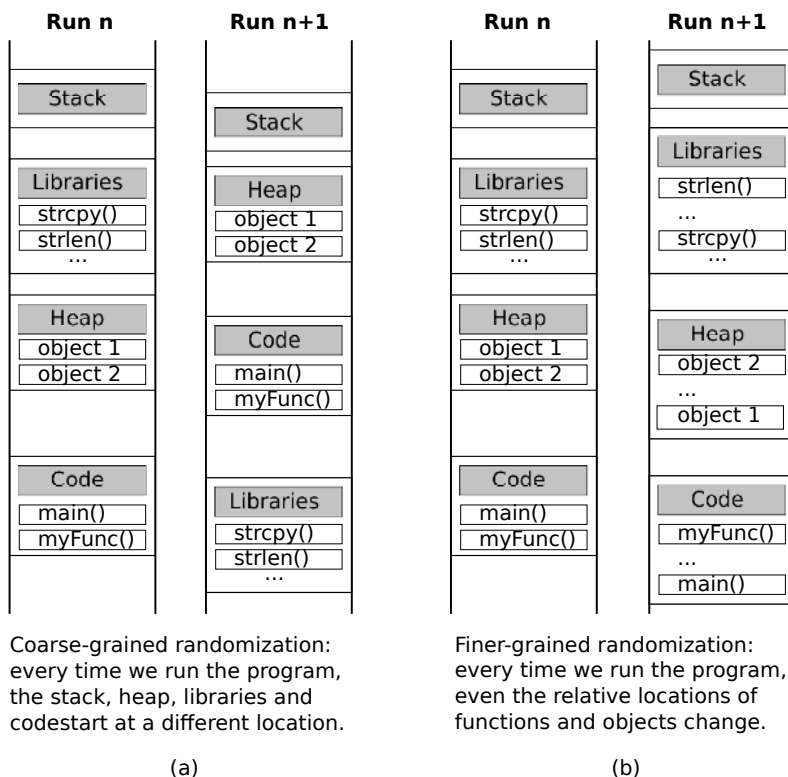


Figure 9-33. Address-space layout randomization: (a) coarse-grained (b) fine-grained

and parcel of mainstream operating systems such as Windows since 2017 and is supported by many compiler toolchains.

To ensure that the control flow in the program always follows legitimate paths during execution, CFI analyzes the program in advance to determine what the possible legitimate targets are for jump instructions, call instructions, and return instructions. For calls and jumps, it only considers those that may be tampered with by an attacker. If the code contains an instruction such as `call 0x543210`, there is nothing the attacker can do—the instruction will *always* call the function at that address. Since the code segment is read-only, it is difficult for the attacker to modify the call target. However, suppose the instruction is an *indirect* call, such as `call fptr`, where `fptr` is a function pointer stored at some location in memory. If the attacker can change that memory location, for instance using a buffer overflow, she can control which code will be executed. You may wonder what the legitimate targets for such indirect calls are. A simple if crude answer is: all the functions of which the address was ever stored in a function pointer. This is generally a very small fraction of the functions in the program. Restricting the indirect calls (calls

with function pointers) such that they may only target the entry points of those functions, raises the bar for attackers considerably. If more security is needed, we could refine the set further, for instance by demanding that the number and types of the arguments in the caller and callee match also.

Given the sets of legitimate targets for indirect calls, indirect jumps, and returns, we now rewrite the code to ensure that such calls, jumps, and returns only use these targets. There are many ways to do so. A simple solution is shown in pseudo assembly code in Fig. 9-34. In the figure, the original code for 3 functions without CFI is on the left. The main function stores the addresses of *foo()* and *bar()* in function pointers and then uses these function pointers to call the functions. The code on the right shows the instrumented versions of these functions, with CFI. The instrumentation starts by assigning each set of legitimate target addresses a label. For instance, the set of legitimate targets for indirect calls gets an 8B label *L1*, the set for indirect jumps gets the label *L2* (not used in the example), and the set for returns gets the label *L3*. These labels are then stored in front of the target addresses in the set (Lines 1, 11, 29, and 32). Finally, instrumentation is added to check each indirect call, jump, and return to see if the address they target has the required label (Lines 7–9, 17–19, 27–28, and 30–31). Let us take Lines 7–9 as an example. Instead of a regular *ret* instruction which takes the return address off the stack and jumps to it in one go, the instrumented code explicitly pops the return address into a register, checks whether the label is a valid return label, and then jumps to the instruction following the 8B label. The case for indirect calls is similar: the code in Line 27–28 checks whether the memory location just before the function about to be called has the correct label and if so, does the indirect call.

While the above CFI scheme severely constrains the attackers' actions, it is not foolproof. For instance, by overwriting the return address, the attacker can still steer the program to *any* call site. For better security, we could make the sets of target addresses as small as possible and perhaps even keep track explicitly of the actual call site (e.g., in a separate shadow stack out of reach of the attacker). Indeed, security researchers have proposed many flavors of fine-grained CFI, but most of them are never applied in practice.

9.8.3 Access Restrictions

Isolating security domains such as the operating system and user processes from each other is one of the cornerstones of security. In the absence of software or hardware vulnerabilities, protection rings ensure that no data in the operating system kernel is accessible to user processes. Inside the security domain, we can further partition code from data using data execution protection. In a nutshell, if memory areas are executable, they should not be writable, and if they are writable, they should not be executable. This is known as data execution prevention, a topic we covered in Sec. 9.5.1. Similarly, access control lists and capabilities determine who can do what with which resources. All of these access restrictions help to draw up

1 foo:	L1 /* label: legitimate indirect call target (8B) */
2 instr_1 /* start of foo() */	foo:
3 instr_2	instr_1 /* start of foo() */
4 instr_3	instr_2
5 ...	instr_3
6 ret	...
7	pop reg0 /* store return addr in reg0 */
8	if (*reg0 != L3) raise_alarm() /* check label*/
9	else jmp (reg0 + 8)
10	
11 bar:	L1 /* label: legitimate indirect call target (8B) */
12 instr_1 /* start of bar() */	bar:
13 instr_2	instr_1 /* start of bar() */
14 instr_3	instr_2
15 ...	instr_3
16 ret	...
17	pop reg0 /* store return addr in reg0 */
18	if (*reg0 != L3) raise_alarm() /* check label*/
19	else jmp (reg0 + 8)
20	
21 main:	main:
22 instr1	instr1
23 ...	...
24 fptr1 = foo	fptr1 = foo
25 fptr2 = bar	fptr2 = bar
26 ...	
27 fptr1() ; indirect call to foo()	if (*(fptr1-8) != L1) raise_alarm()
28 fptr2() ; indirect call to bar()	else (fptr1)()
29 instr_21	L3 /* label: legitimate target for return */
30 instr_22	if (*(fptr2-8) != L1) raise_alarm()
31 ...	else (fptr1)()
32	L3 /* label: legitimate target for return */
33	instr_21
34	instr_21

(a) (b)

Figure 9-34. Control-flow integrity (pseudo-code): (a) without CFI, (b) with CFI.

strong walls between attackers and the system's crown jewels, in accordance with the security principles by Saltzer and Schroeder.

While it is intuitively clear why we must protect the operating system from untrusted user processes, we now argue that stopping the operating system from accessing code or data in user processes is also useful. Our first example concerns operating systems that map the kernel into the address space of every process so that a system call does not require an address space switch. Linux is such an operating system (except on older CPUs vulnerable to Meltdown, where KPTI separates the kernel and user address spaces). In that case, bugs such as null pointer dereferences (see Sec. 9.5.4) become much more serious because the kernel may execute user memory.

It would be better if the kernel did not have a way to execute code in the user process, accidentally or not. It probably should not be able to read from user data either, because that would allow an attack to feed malicious data to the operating system. To prevent inadvertent execution of user code in the kernel, many CPUs today implement what Intel calls **SMEP (Supervisor Mode Execution Protection)** and **SMAP (Supervisor Mode Access Protection)**. When SMEP and SMAP are enabled, all attempts to execute (SMEP) or access (SMAP) memory in user processes from the operating system kernel result in a fault. But wait! What if the kernel really does need to access some memory in a user process, for instance to read or write a buffer to send over the network? In that case, the kernel can temporarily disable the SMAP restrictions and do whatever needs to be done and then turn the restrictions back on again.

Our second example of the need to restrict accesses by the operating system to user memory is that sometimes, on rare occasions, we do not trust the operating system. This may sound very strange. Did we not build our model of the trusted computing base around the operating system, with rings of protection and supervisor mode and all that? Well yes, but there are still situations where even the kernel of the operating system is not part of the TCB for the application. Suppose the Coca Cola Company wants to run simulations in a cloud environment to develop a new recipe for its Coca Cola syrup. The actual Coca Cola Formula is probably the most famous trade secret in the world. In 1919, the only written copy of the formula was placed in a vault in a bank. In 2011, it was moved to another vault in Atlanta, where for a small fee, visitors can go and stare at it (the vault, not the formula). Any computation on the formula in the cloud would be extremely sensitive and the company might be a tad disappointed if a sysadmin of the cloud provider were to post a message on Twitter saying:

'Sup peeps! I hacked our operating system to learn the Coca Cola Formula. Here it is. LOL.

While it is unlikely that the Coca Cola Company would use a public cloud for the most jealously guarded trade secret in history, a lot of organizations do process sensitive data in the cloud or use secret algorithms that should not leak, not even if

the hypervisor or operating system is hacked, the sysadmin is bribed, or the cloud provider proves untrustworthy.

Similarly, if an organization provides an smartphone app that is important to the customer and a high-value target for attackers, it may not trust the operating system on a user's smartphone. For instance, banking apps do not want an iPhone, even if completely compromised, to be able to steal from their customers.

Neither SMEP nor SMAP will help here. After all, the operating system itself is not trusted and may turn off any restriction as it sees fit. We need something that even the operating system can touch.

For this reason, CPU vendors have developed CPU extensions called **TEE (Trusted Execution Environments)**. A TEE is a secure "enclave" on your CPU where you can perform secret computations on sensitive data and the hardware guarantees that not even operating systems can access them. For instance, ARM TrustZone is a security extension for ARM processors that allows the CPU to switch between two worlds: normal world and secure world. The regular operating systems (e.g., Linux) and all the regular applications run in normal world. If the operating system cannot be trusted, the applications in normal world are toast. However, the applications in secure world are still safe.

Applications that care a lot about security, such as banks or companies selling fizzy drinks, can run a small part of their functionality in the TEE (e.g., their wallet or the code processing their secret formula). Some TEEs even have a separate, minimalistic, secure operating system to run these trusted (parts of) applications. The processor enters a secure world by means of a special instruction which operates a bit like a system call: by executing the instruction, the CPU traps into the secure world to perform the appropriate service. While the applications in the secure world may access all of the memory, the TEE's memory is physically protected from all accesses by code running in normal world. After doing whatever needs to be done, the trusted applications transition back to normal world.

There is a lot more to say about TEEs and Confidential Computing. Each vendor has its own solution and some vendors even have more than one. To name an example, Intel initially deployed a solution called **SGX (Software Guard Extension)** and when that turned out to be vulnerable to microarchitectural attacks, pushed an improved design called **TDX (Trust Domain Extensions)** which caters more to virtualization. There are significant differences between the different TEEs. For instance, some TEEs do not run a separate operating system while others do. These topics are beyond this book. We just want you to be aware of their existence and that they are used to implement what is now known as **Confidential Computing**. TEEs have been a mixed success, as researchers found various vulnerabilities in the design and implementation of both hardware and software. It appears that security is hard to get right even if multibillion dollar chip vendors set out to develop features with the explicit goal of doing so.

It should come as no surprise that many of the vulnerabilities in TEEs were related to transient execution and side channel attacks. Given how indirect these

attacks are, is there any hope of stopping them? The answer is: it depends. In general, whenever security domains share resources, there is a risk of side channels. Saltzer and Schroeder's Principle of Least Common Mechanism suggests that we have as little as possible of it. Unfortunately, modern computer systems share resources all over the place: cores, caches, TLBs, memory, branch predictors, buses, etc. However, that does not mean that we are powerless. If the operating system is able to either partition the resources, or flush their state between the execution of different security domains, life becomes much harder for attackers.

For instance, by sacrificing some efficiency, operating systems are sometimes able to partition resources such as caches even at a fine granularity. A well-known technique, known as **page coloring**, is an example of such partitioning of the cache, which works by giving different security domains memory pages that map to disjoint cache sets. As a simple example, imagine that the operating system gives process 1 only pages that map onto cache sets $0 - (N - 1)$ and process 2 pages that map onto cache sets $(N - 1) - M$. Whatever the cache activity in process 1, it will not normally affect the cache activity of process 2. Nowadays, rather than relying on the operating system contorting itself during memory allocation, cache partitioning is sometimes also supported by hardware. For instance, Intel's **CAT (Cache Allocation Technology)** allows one to set aside a number of ways of an n -way set associate cache.

9.8.4 Code and Data Integrity Checks

Some operating systems reduce the number of bugs in the operating system by accepting only drivers and other code that are signed by trusted vendors with a digital signature. Such **driver signing** helps to ensure a measure of quality of operating system extensions. A similar mechanism is commonly used for updates: only signed updates from a trusted source will be installed. Taking the idea one step further, operating systems such as Windows may completely "lock down" the machine to ensure that it can run only trusted software in general. In that case, it becomes very difficult to run non-authorized apps of whatever nature even for malware that manages to obtain elevated privileges, because the check is performed in a hardware-protected environment that the malware cannot easily bypass.

Finally, many modern operating systems offer functionality to ensure that the code to check the signatures, the operating system itself, and indeed all steps involved in the booting process were loaded correctly. The verification takes a number of steps, just like the boot process itself takes multiple steps.

To secure boot a machine, we need a root of trust, typically a secure hardware device, to get the ball rolling. The procedure is roughly as follows. A microcontroller starts the boot process by executing a small amount of firmware from a ROM (or flash memory that cannot be reprogrammed by an attacker). As we have seen in Sec. 1.3, the UEFI firmware then loads a bootloader, which in turn loads the operating system. As shown in Fig. 9-35(a), a secure boot process checks all of

these stages. For instance, the UEFI firmware will protect the integrity of the bootloaders by checking their signatures using the key information embedded in the firmware. Boot loaders or drivers without the appropriate signatures will never even run. In the next stage, the bootloader checks the signature of the operating system kernel. Again, unless the signature is correct, the kernel will not run. Finally, the other components of the kernel, as well as all the drivers are checked by the kernel in a similar fashion. All attempts to change any component in any stage in the boot process will lead to a verification error. And the verification chain need not stop there. For instance, the operating system may start an anti-malware program to check all subsequent programs.

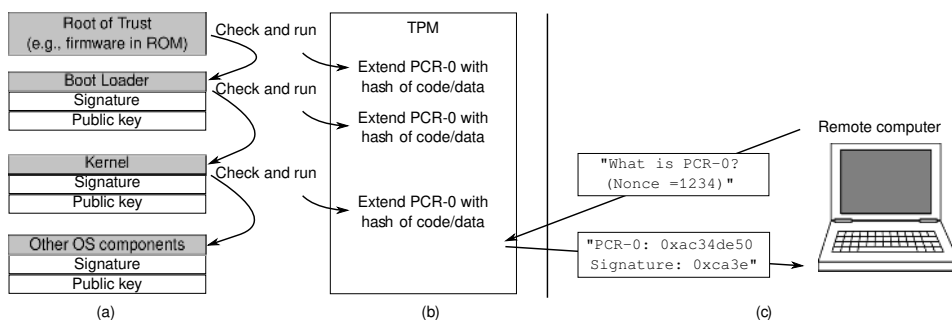


Figure 9-35. Securing and verifying the boot process.

9.8.5 Remote Attestation Using a Trusted Platform Module

Let us now go back to the issue of remote attestation and TPM and see how they fit in the store. The question was: once the operating system is booted, how can we tell if it was booted correctly and securely? Whatever is on the screen cannot necessarily be trusted. After all, the attacker could have installed a new operating system that displays whatever the attacker wants it to display. To verify that a system was booted in the appropriate manner, we can use remote attestation. The idea is that we use another computer to verify the trustworthiness of a target machine. Special cryptographic hardware on that we called trusted platform module on the machine that needs to be verified, allows it to prove to a remote party that all the right steps in the secure boot process were taken. The TPM has a number of Platform Configuration Registers (PCR-0, PCR-1, ...) that are set to a known value on every boot. Nobody can write directly to these registers. The only thing one can do is “extend” it. In particular, if you ask the TPM to extend the PCR-0 register with a value X , it will calculate a hash of the concatenation of the current value PCR-0 and the value X and store the result in PCR-0. By extending the value in PCR-0 with new values, you get a hash chain of arbitrary length.

By integrating the TPM in our secure boot process above, we can create proof that the machine was at least booted in a secure way. The idea is that the booting computer creates “measurements”—hashes of what is loaded in memory. For instance, Fig. 9-36(b) shows that whenever the firmware has checked the signature of a bootloader, it asks the TPM to extend PCR-0 with a hash of the code and data it loaded in memory. For simplicity, we will assume that the computer uses only PCR-0. When the bootloader starts running, it does the same with the kernel image, and the kernel, once it is running, does the same with the other operating system components. In the end, having the right value in PCR-0 proves that the boot process was executed correctly and securely. The resulting cryptographic hash in PCR-0 serves as a hash chain that binds the kernel to the bootloader and the bootloader to the root of trust.

As shown in Fig. 9-36(c), the remote computer now verifies if this is the case by sending an arbitrary number known as a *nonce* (of, say, 160 bits) to the TPM and asking it to return (a) the value of PCR-0, and (b) a digital signature of the concatenation of the value of PCR-0 and the nonce. The TPM signs them with its (unique and unforgeable) private Attestation Identity Key. The corresponding public key is well known, so the remote computer can now (a) verify the signature and (b) check that the PCR-0 is correct (proving that the right steps were taken in the boot process). In particular, it first checks the signature and the nonce. Next, it looks up the three hashes in its database of trusted bootloaders, kernels, and OS components. If they are not there, the attestation fails. Otherwise, the challenging party recreates the combined hash of all three components and compares it to the value of PCR-0, as received from the attesting side. If the values are the same, the remote party knows that the attesting machine was booted in a trusted way. The signed result prevents attackers from forging the result, and since we know that the trusted bootloader performs the appropriate measurement of the kernel and the kernel in turn measures the application, no other code configuration could have produced the same hash chain. In case you are wondering about the role of the nonce: it ensures that the signature is “fresh”—making it impossible for the attacker to send an old, recorded reply.

9.8.6 Encapsulating Untrusted Code

Viruses and worms are programs that get onto a computer without the owner’s knowledge and against the owner’s will. Sometimes, however, people more-or-less intentionally import and run foreign code on their machines. In the past, Web browsers would execute Java “Applets” in the browser and Microsoft even allowed the execution of native code. Neither of these solutions will be missed by many. Nowadays, we still execute untrusted code in our browsers, for instance, in the form of JavaScript or other scripting languages.

Even operating systems sometimes allow foreign code to be executed in the kernel. An example includes Linux’ **eBPF (extended Berkeley Packet Filter)**,

which allows users to write programs to perform such tasks as high-performance filtering of network packets. The code you write executes inside the operating system itself.

It should be clear that allowing foreign code to run on your machine is more than a wee bit risky. Nevermind running it directly inside the operating system. Nevertheless, some people do want to run such code, so the question arises: “Can such code be run safely”? The short answer is: “Yes, but not easily.” The fundamental problem is that when a process (or operating system) imports untrusted code into its address space and runs it, that code is running with all the power the user (or operating system) has, including the ability to read, write, erase, or encrypt the user’s disk files, email data to far-away countries, and much more.

Long ago, operating systems developed the process concept to build walls between users. The idea is that each process has its own protected address space and its own UID, allowing it to touch files and other resources belonging to it, but not to other users. For providing protection against one *part* of the process (the foreign code) and the rest, the process concept does not help. Threads allow multiple threads of control within a process, but do nothing to protect one thread against another one.

One solution is to run the foreign code as a separate process, but this is not always what we want. For instance, eBPF really wants to run inside the kernel. Various methods of dealing with foreign code have been implemented. Below we will look at one such method: sandboxing. In addition, code signing can also be used to verify the source of the foreign program.

Sandboxing

The aim of **sandboxing** is to confine the untrusted/foreign code to a limited range of virtual addresses enforced at run time (Wahbe et al., 1993). It works by dividing the virtual address space up into equal-size regions, which we will call **sandboxes**. Each sandbox must have the property that all of its addresses share some string of high-order bits. For a 48-bit address space, we could divide it up into many sandboxes on 4-GB boundaries so that all addresses within a sandbox have a common upper 16 bits. Equally well, we could have a few more sandboxes on 1-GB boundaries, with each sandbox having an 18-bit address prefix. The sandbox size should be chosen to be large enough to hold the largest chunk of foreign code without wasting too much virtual address space. Physical memory is not an issue if demand paging is present, as it usually is. Each foreign program is given two sandboxes, one for the code and one for the data, as illustrated in Fig. 9-36(a). For illustration purposes, we will assume a small machine with 256-MB and 16 sandboxes of 16 MB each.

The basic idea behind a sandbox is to guarantee that the untrusted code, which we will refer to as “foreign program” from now on, cannot jump to code outside its code sandbox or reference data outside its data sandbox. The reason for having

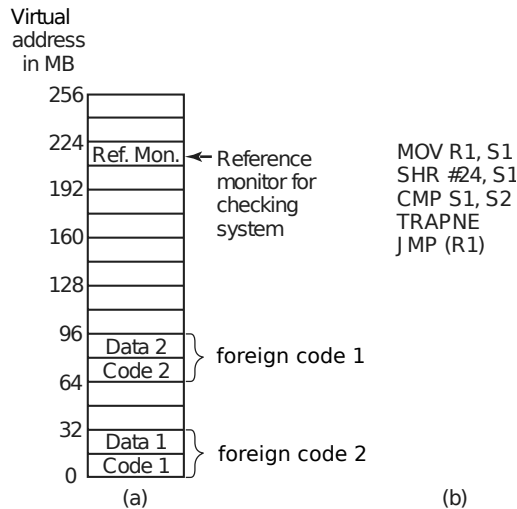


Figure 9-36. (a) Memory divided into 16-MB sandboxes. (b) One way of checking an instruction for validity.

two sandboxes is to prevent a foreign program from modifying its code during execution to get around these restrictions. By preventing all stores into the code sandbox, we eliminate the danger of self-modifying code. As long as a foreign program is confined this way, it cannot damage the browser or other programs, plant viruses in memory, or otherwise do any damage to memory.

As soon as a foreign program is loaded, it is relocated to begin at the start of its sandbox. Then checks are made to see if code and data references are confined to the appropriate sandbox. In the discussion below, we will just look at code references (i.e., `JMP` and `CALL` instructions), but the same story holds for data references as well. Static `JMP` instructions that use direct addressing are easy to check: does the target address land within the boundaries of the code sandbox? Similarly, relative `JMP`s are also easy to check. If the foreign program has code that tries to leave the code sandbox, it is rejected and not executed. Similarly, attempts to touch data outside the data sandbox cause the foreign program to be rejected. This is the easy part.

The hard part is dynamic/indirect `JMP` instructions. As we saw in our discussion of CFI, most machines have an instruction in which the address to jump to is computed at run time, put in a register, and then jumped to indirectly, for example by `JMP (R1)` to jump to the address held in register 1. The validity of such instructions must be checked at run time. This is done by inserting code directly before the indirect jump to test the target address. An example of such a test is shown in Fig. 9-36(b). Remember that all valid addresses have the same upper k bits, so this prefix can be stored in a scratch register, say `S2`. Such a register

cannot be used by the foreign program itself, which may require rewriting it to avoid this register.

The code works as follows: First the target address under inspection is copied to a scratch register, S1. Then this register is shifted right precisely the correct number of bits to isolate the common prefix in S1. Next the isolated prefix is compared to the correct prefix initially loaded into S2. If they do not match, a trap occurs and the foreign program is killed. This code sequence requires four instructions and two scratch registers.

Patching the binary program during execution requires some work, but it is doable. It would be simpler if the foreign program were presented in source form and then compiled locally using a trusted compiler that automatically checked the static addresses and inserted code to verify the dynamic ones during execution. Either way, there is some runtime overhead associated with the dynamic checks. Wahbe et al. (1993) have measured this as about 4%, which is perhaps not too bad.

A second problem that must be solved is what happens when a foreign program tries to make a system call. The solution here is straightforward. The system-call instruction is replaced by a call to a special module called a **reference monitor** on the same pass that the dynamic address checks are inserted (or, if the source code is available, by linking with a special library that calls the reference monitor instead of making system calls). Either way, the reference monitor examines each attempted call and decides if it is safe to perform. If the call is deemed acceptable, such as writing a temporary file in a designated scratch directory, the call is allowed to proceed. If the call is known to be dangerous or the reference monitor cannot tell, the foreign program is killed. If the reference monitor can tell which foreign program called it, a single reference monitor somewhere in memory can handle the requests from all foreign programs. The reference monitor normally learns about the permissions from a configuration file.

9.9 RESEARCH ON SECURITY

Few topics see more activity than security in operating systems. Research is taking place in all areas: cryptography, attacks, defenses, compilers, hardware, formal methods, etc. A more-or-less continuous stream of high-profile security incidents ensures that research interest in security, both in academia and in industry, is not likely to waver in the next few years either.

Even venerable concepts such as passwords are still active research topics, for instance when modeling the strength of a password (Pasquini et al., 2021) or protecting password vaults as used by password managers using a set of decoy vaults (Cheng et al., 2021). Also other authentication factors, such as security keys, are targeted by security researchers to probe for weaknesses (Roche et al., 2021).

Bugs in the operating system itself are clearly a major concern and researchers are developing testing frameworks for various operating systems to find and